

Réponses aux questions

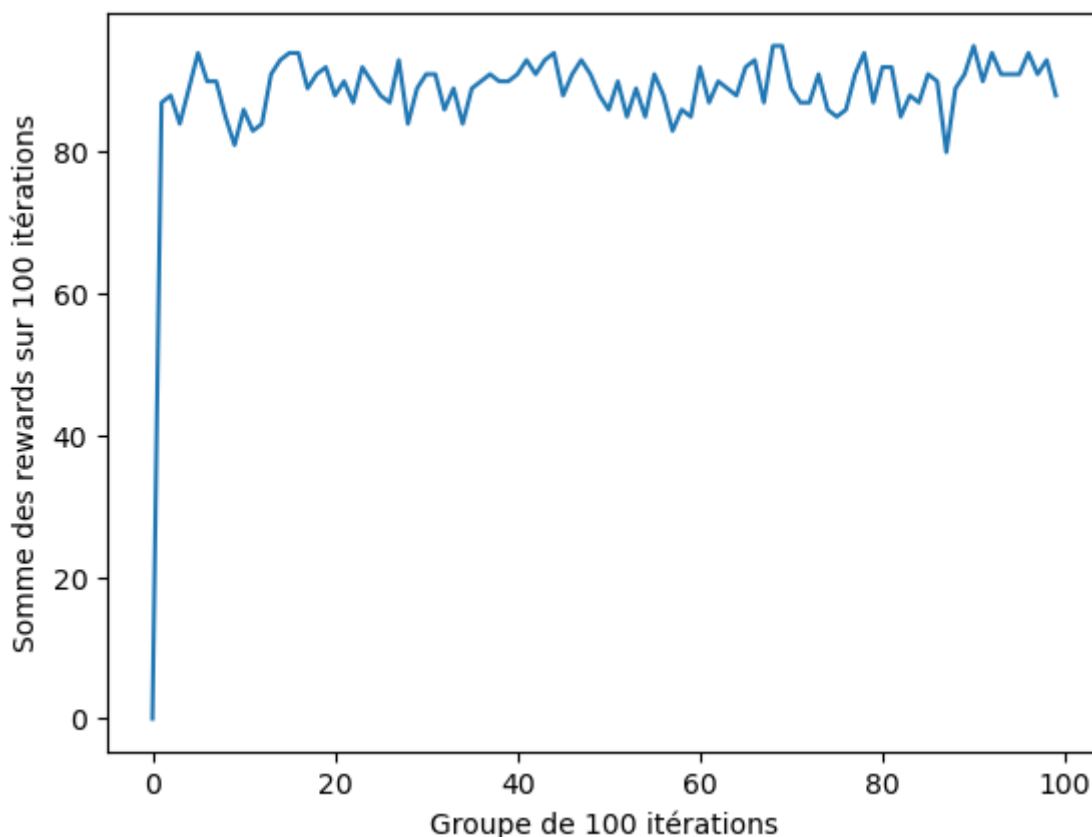
FROZEN LAKE PROBLEM

Identify and copy the python code that performs the value function adaptation (i.e., the modification of Q-values). Is it the same as the one presented in the lesson ? does it correspond to SARSA or Q-learning ? what is the difference between those learning algorithms ? Explain.

```
new_value = ((1 - alpha) * old_value) + (alpha * (reward + gamma * next_max))  
q_table[state, action] = new_value
```

C'est la formule d'adaptation de l'algorithme Q-Learning. La différence entre les algorithmes SARSA et Q-Learning se situe au niveau de la prochaine Q-value qui est prise. SARSA va choisir la Q-Value la plus probable étant donné sa politique alors que Q-Learning va prendre la Q-Value maximale du prochain état.

Using a 4x4 environment modify the code to stop the learning process every 100 epochs to evaluate the performance of the agent. To do this, you can run 100 episodes (Each time the agent is placed in a starting position and tries to reach the target. Take note of how many times it reaches the target) while putting alpha to zero (no learning) and epsilon to zero (no exploration). Generate a plot of the evolution of the performance as a function of iterations.



Modify gamma (the discount factor) to 0.9, and alpha (learning rate) to 0.05 and 0.01. Compare the resulting plots of performance vs. epochs of the 6 combinations of hyper-parameters and provide your observations.

Nous avons modifié les paramètres de départ pour effectuer un total de 300 epochs et enregistrer les performances toutes les 5 epochs. Nous avons fait cela car avec 10000 epochs et un enregistrement toutes les 100 epochs, nous ne voyions rien d'intéressant sur les graphiques.

Après avoir généré les 6 graphiques, on observe que les résultats sont très aléatoires. Visiblement, dès que le modèle trouve le chemin, le score devient très élevé et le reste sans vraiment augmenter ou baisser. C'est très "binaire". Cela est en partie dû au fait que l'agent commence toujours dans les mêmes conditions initiales, il y a donc une "bonne solution" très facile à trouver.

Run a 5x5 environment. Track the learning process to determine the minimum number of epochs needed to allow the agent to have a good performance. Can it reach a perfect performance ? Is it more rapid than the exhaustive search? Explain.

L'entraînement prend plus de temps mais les résultats sont similaires. Dès que l'agent a trouvé la solution, le score monte à ~90 et reste autour de cette valeur. Oui l'apprentissage reste quand même beaucoup plus rapide que l'apprentissage par brute-force. Cependant, l'agent n'atteint pas de performance "parfaite" car il y a une certaine part d'exploration, donc un certain aléatoire.

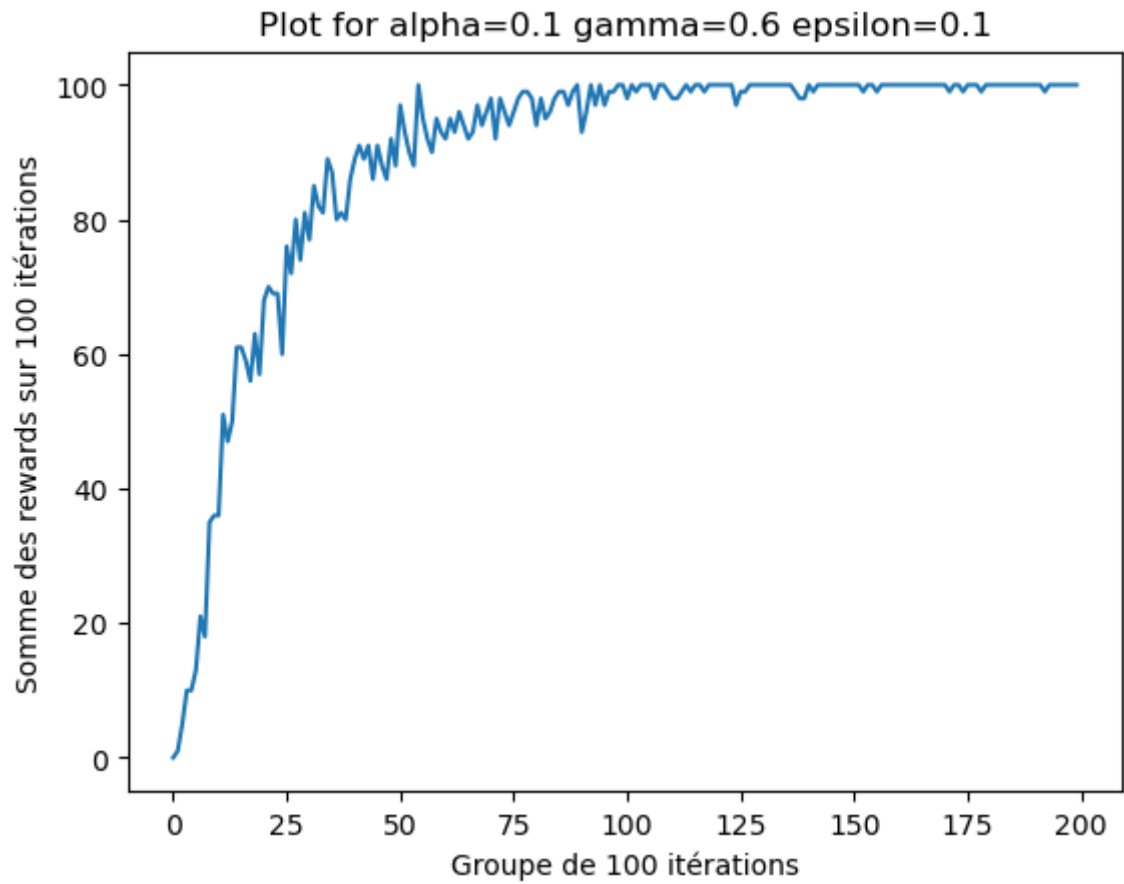
Try to modify the reward function by providing the agent with a punishment of $r=-0.1$ for each step it takes to try to reach the target instead of giving it a reward $r=+1$ when it reaches the target.

C'est une manière alternative d'entraîner l'agent. Dans tous les cas, il maximise son reward en étant à l'arrivée. Cette approche est peut-être meilleure dans le sens où il a intérêt à y arriver rapidement.

TAXI PROBLEM

Using the taxi environment modify the code to stop the learning process every 100 epochs to evaluate the performance of the agent. To do this, you can run 100 episodes (Each time the agent is placed in a starting position and tries to reach the target. Take note of how many times it reaches the target) while putting alpha to zero (no learning) and epsilon to zero (no exploration). Generate a plot of the evolution of the performance as a function of iterations.

Nous avons repris la fonction créée dans le premier notebook et avons obtenu le graphique suivant :



Perform hyper-parameter tuning considering the value of epsilon (exploration), the discount factor and the number of epochs. Present your results.

Voici la liste de tous les hyperparamètres testés :

Score: 100.00	Epochs: 5000	Epsilon: 0.1	Gamma: 0.9
Score: 100.00	Epochs: 5000	Epsilon: 0.2	Gamma: 0.9
Score: 100.00	Epochs: 5000	Epsilon: 0.3	Gamma: 0.6
Score: 100.00	Epochs: 5000	Epsilon: 0.3	Gamma: 0.9
Score: 100.00	Epochs: 5000	Epsilon: 0.4	Gamma: 0.6
Score: 100.00	Epochs: 5000	Epsilon: 0.4	Gamma: 0.9
Score: 100.00	Epochs: 10000	Epsilon: 0.1	Gamma: 0.6
Score: 100.00	Epochs: 10000	Epsilon: 0.1	Gamma: 0.9
Score: 100.00	Epochs: 10000	Epsilon: 0.2	Gamma: 0.6
Score: 100.00	Epochs: 10000	Epsilon: 0.2	Gamma: 0.9
Score: 100.00	Epochs: 10000	Epsilon: 0.3	Gamma: 0.4
Score: 100.00	Epochs: 10000	Epsilon: 0.3	Gamma: 0.6
Score: 100.00	Epochs: 10000	Epsilon: 0.3	Gamma: 0.9
Score: 100.00	Epochs: 10000	Epsilon: 0.4	Gamma: 0.4
Score: 100.00	Epochs: 10000	Epsilon: 0.4	Gamma: 0.6
Score: 100.00	Epochs: 10000	Epsilon: 0.4	Gamma: 0.9
Score: 100.00	Epochs: 15000	Epsilon: 0.1	Gamma: 0.4
Score: 100.00	Epochs: 15000	Epsilon: 0.1	Gamma: 0.9
Score: 100.00	Epochs: 15000	Epsilon: 0.2	Gamma: 0.4
Score: 100.00	Epochs: 15000	Epsilon: 0.2	Gamma: 0.6
Score: 100.00	Epochs: 15000	Epsilon: 0.2	Gamma: 0.9
Score: 100.00	Epochs: 15000	Epsilon: 0.3	Gamma: 0.4
Score: 100.00	Epochs: 15000	Epsilon: 0.3	Gamma: 0.6

```

Score: 100.00 | Epochs: 15000 | Epsilon: 0.3 | Gamma: 0.9
Score: 100.00 | Epochs: 15000 | Epsilon: 0.4 | Gamma: 0.4
Score: 100.00 | Epochs: 15000 | Epsilon: 0.4 | Gamma: 0.6
Score: 100.00 | Epochs: 15000 | Epsilon: 0.4 | Gamma: 0.9
Score: 100.00 | Epochs: 20000 | Epsilon: 0.1 | Gamma: 0.4
Score: 100.00 | Epochs: 20000 | Epsilon: 0.1 | Gamma: 0.6
Score: 100.00 | Epochs: 20000 | Epsilon: 0.1 | Gamma: 0.9
Score: 100.00 | Epochs: 20000 | Epsilon: 0.2 | Gamma: 0.4
Score: 100.00 | Epochs: 20000 | Epsilon: 0.2 | Gamma: 0.6
Score: 100.00 | Epochs: 20000 | Epsilon: 0.2 | Gamma: 0.9
Score: 100.00 | Epochs: 20000 | Epsilon: 0.3 | Gamma: 0.4
Score: 100.00 | Epochs: 20000 | Epsilon: 0.3 | Gamma: 0.6
Score: 100.00 | Epochs: 20000 | Epsilon: 0.3 | Gamma: 0.9
Score: 100.00 | Epochs: 20000 | Epsilon: 0.4 | Gamma: 0.4
Score: 100.00 | Epochs: 20000 | Epsilon: 0.4 | Gamma: 0.6
Score: 100.00 | Epochs: 20000 | Epsilon: 0.4 | Gamma: 0.9
Score: 99.00 | Epochs: 15000 | Epsilon: 0.1 | Gamma: 0.6
Score: 98.00 | Epochs: 10000 | Epsilon: 0.2 | Gamma: 0.4
Score: 97.00 | Epochs: 5000 | Epsilon: 0.2 | Gamma: 0.6
Score: 97.00 | Epochs: 10000 | Epsilon: 0.1 | Gamma: 0.4
Score: 92.00 | Epochs: 5000 | Epsilon: 0.1 | Gamma: 0.6
Score: 92.00 | Epochs: 5000 | Epsilon: 0.4 | Gamma: 0.4
Score: 91.00 | Epochs: 5000 | Epsilon: 0.2 | Gamma: 0.4
Score: 89.00 | Epochs: 5000 | Epsilon: 0.1 | Gamma: 0.4
Score: 89.00 | Epochs: 5000 | Epsilon: 0.3 | Gamma: 0.4

```

Find how the state of the agent is computed from the observed variables. For a trained agent list the states where the agent performs the “dropoff” and “pickup” actions and verify if the behavior is the right one. You may use the `env.decode()` function. Explain.

```

dropoff_states = [i for i in range(len(q_table)) if np.argmax(q_table[i]) == 5]
print(f'List of states where the agent has to drop off the passenger:
{dropoff_states}')

pickup_states = [i for i in range(len(q_table)) if np.argmax(q_table[i]) == 4]
print(f'List of states where the agent has to pick up the passenger:
{pickup_states}')

```

Sortie :

```

List of states where the agent has to drop off the passenger: [16, 97, 418, 479]
List of states where the agent has to pick up the passenger: [1, 2, 3, 84, 86, 87,
408, 409, 411, 472, 473, 474]

```

La fonction `env.decode()` permet d'expliquer dans quel état sont les variables pour un état en particulier.

Il existe 500 états différents parce que c'est une grille de 5x5 (25 positions différentes) * 4 destinations possibles * 5 positions du client = 554*5. Il y a 4 états qui sont les correct où déposer le client (bonne endroit

et client dans la voiture) et 12 états où il faut récupérer le client. (pas 16 car si le client est sur la case de destination le jeu est gagné). Après vérification avec la fonction `decode()` ci-dessus, nous avons pu confirmer ces observations.

For a trained agent, use the learned strategy to pickup one by one, three passengers at locations 1, 2 and 3 and drop them at location 0. Perform 100 simulations and for each one, compute the percentage of success and the mean steps required to complete the task.

Le code pour cette question se trouve dans la section "Simulation avec plusieurs passagers" du notebook. Nous avons obtenu un pourcentage de réussite de 96%, avec un nombre moyen d'étapes de 49,06.